PARCIAL – CASO DE ESTUDIO MAINTENANCE PRO

Estudiante: Juan camilo castaño rojas Código: 01250371033

PARTE A – Análisis del dominio

1. Identificación de clases

Se identificaron 14 clases a partir del texto y de las reglas:

Clase	Responsabilidad principal
Empresa	Agrupa los equipos y usuarios de una organización cliente y permite registrar y localizar sus equipos.
Equipo	Representa la máquina industrial: guarda sus datos de identificación y controla su propio estado (OPERATIVO, EN_MANTENIMIENTO, FUERA_DE_SERVICIO).
ReporteFalla	Registra la falla detectada por un operador (fecha, descripción, prioridad, equipo) y garantiza que origine como máximo una orden (RN3).
OrdenMantenimiento	Coordina el trabajo de mantenimiento: gobierna su ciclo de vida (RN5), exige sus reglas (RN1, RN4, RN6) y calcula horas y costo (RN9).
Actividad	Documenta un trabajo realizado en la orden: qué se hizo, cuántas horas y qué técnico lo hizo.
Repuesto	Elemento del catálogo (código, nombre, costo de referencia) que existe independientemente de las órdenes.
RepuestoUtilizado	Registra el uso de un repuesto en una orden: cantidad y costo unitario vigente al momento de usarlo (RN8).
Usuario (abstracta)	Generaliza a las personas de la plataforma: identificación, nombre, correo y rol.
Operador	Detecta fallas y las reporta.
Supervisor	Revisa reportes, genera y cancela órdenes y decide el destino del equipo.
Tecnico	Ejecuta órdenes como responsable o apoyo, registra actividades y conoce su propia agenda (RN2).
GestorMantenimiento («Sistema»)	Fachada que recibe las solicitudes del supervisor y coordina a los objetos del dominio.
ServicioExterno, Notificador, ProveedorRepuestos (interfaces)	Contratos para integrar servicios externos (correo, proveedores de repuestos, y a futuro formas de pago) sin modificar el dominio.
NotificadorCorreo, ProveedorRepuestosHttp	Implementaciones concretas de las interfaces anteriores.

Sustantivos descartados como clase

Sustantivo del texto	Decisión	Por qué
código interno, tipo, marca, modelo, fecha de adquisición	Atributos de Equipo	Son datos simples sin comportamiento ni identidad propia.

Sustantivo del texto	Decisión	Por qué
estado, prioridad	Enumeraciones	Son conjuntos cerrados de valores sin comportamiento propio (EstadoEquipo, EstadoOrden, Prioridad); una clase con identidad sería excesiva.
horas trabajadas	Atributo de Actividad	Es un número que describe una actividad; no tiene ciclo de vida propio.

2. Responsabilidades

Se seleccionaron Equipo, ReporteFalla, OrdenMantenimiento y Tecnico. Cada responsabilidad se corresponde con un método del diagrama de clases y del código.

Clase	Responsabilidad	Método(s)
Equipo	1. Pasar a EN_MANTENIMIENTO cuando una orden entra en ejecución.	poner_en_mantenimiento()
	2. Cerrar el mantenimiento según la decisión del supervisor y rechazar destinos inválidos.	finalizar_mantenimiento(destino)
	3. Informar si está disponible para operar.	esta_operativo()
ReporteFalla	1. Conservar los datos de la falla (fecha, descripción, prioridad, equipo, operador).	constructor y atributos
	2. Informar si ya originó una orden.	tiene_orden()
	3. Asociar su única orden y rechazar un segundo intento.	asociar_orden(orden)
OrdenMantenimiento	1. Gobernar su ciclo de vida: transiciones válidas, guardas y permisos.	programar(), iniciar(), suspender(), reanudar(), cancelar(), finalizar()
	2. Registrar el trabajo realizado: actividades y repuestos.	registrar_actividad(), agregar_repuesto()
	3. Calcular horas totales y costo de la orden.	horas_totales(), costo_total()
Tecnico	1. Mantener su agenda de órdenes como responsable.	registrar_orden(), retirar_orden()
	2. Indicar si tiene una orden programada en una fecha.	orden_programada_en(fecha)
	3. Informar su disponibilidad en una fecha.	esta_disponible(fecha)

3. Decisión de diseño: separar Orden, Actividad y RepuestoUtilizado

Guardar todo dentro de OrdenMantenimiento obligaría a tener listas de datos mezclados en una sola clase. Conviene separarlas por estas razones:

- **Cardinalidad.** Una orden tiene 0..* actividades y 0..* repuestos utilizados, y ambos conjuntos crecen de forma independiente (una orden puede tener cinco actividades y un solo repuesto). Si todo viviera en la orden, habría “grupos repetidos” sin estructura (por ejemplo, listas paralelas de descripciones, horas y cantidades), lo que rompe la primera forma normal y complica validar y consultar.

- **Cada elemento tiene datos y reglas propias.** Una Actividad tiene descripción, horas, fecha y técnico, y valida que sus horas sean positivas. necesita contar actividades y sumar sus horas, algo natural con una colección de objetos y torpe con campos sueltos.

- **el costo histórico.** RepuestoUtilizado es la clase que "congela" el costo_unitario vigente al usar el repuesto, mientras Repuesto (catálogo) conserva solo el costo de referencia actual. Si la orden guardara únicamente una referencia al catálogo, un cambio de precio alteraría costo_total() de órdenes ya cerradas y se perdería la trazabilidad contable. Además, cantidad y costo_unitario son atributos de la relación orden–repuesto (de muchos a muchos), que se modela con una clase propia.

- **Cohesión (principio de responsabilidad única).** OrdenMantenimiento se concentra en su ciclo de vida y sus reglas; el detalle del trabajo realizado queda en clases pequeñas y fáciles de probar.

- **Ciclo de vida.** Con composición, actividades y repuestos utilizados se crean y se eliminan con la orden, sin objetos huérfanos.

Ejemplo comprobado en la ejecución del código (Parte E): la orden 1 usa 2 filtros a $45.000; después el catálogo sube el filtro a $60.000, pero costo_total() sigue en $90.000.

PARTE B – Diagrama de clases UML

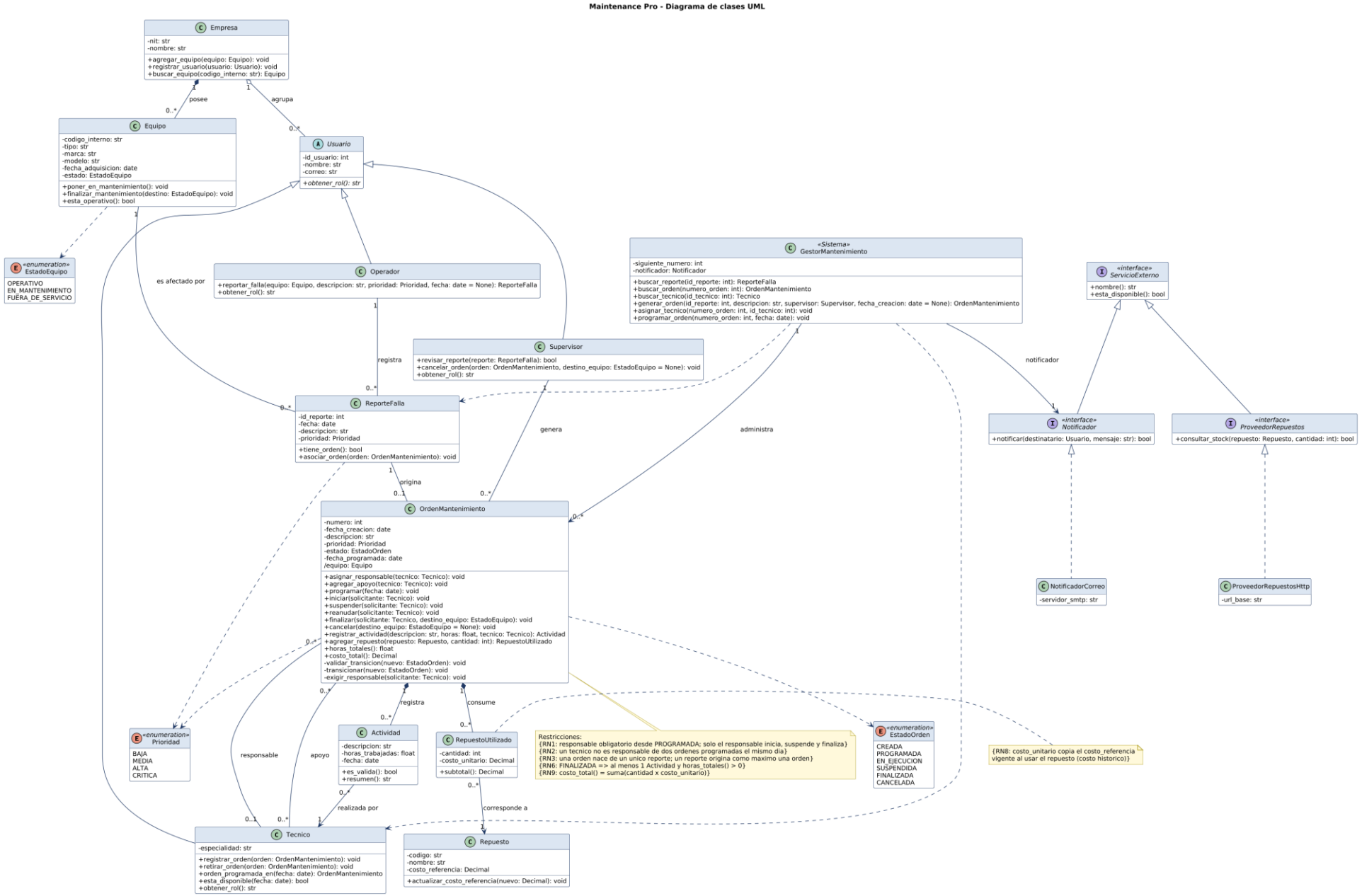
4. Modelo estructural

Requisito	Dónde se ve en la Figura 1
Relaciones entre Empresa, Equipo, ReporteFalla, OrdenMantenimiento, Tecnico, Actividad y Repuesto	Empresa — Equipo; Equipo — ReporteFalla; ReporteFalla — OrdenMantenimiento; OrdenMantenimiento — Tecnico (responsable y apoyo); OrdenMantenimiento — Actividad; OrdenMantenimiento — RepuestoUtilizado → Repuesto.
(a) Jerarquía de usuarios con generalización	Usuario (abstracta, con obtener_rol() abstracto) — Operador, Supervisor, Tecnico.
(b) Abstracción para servicios externos y formas de pago	Interfaz ServicioExterno, especializada por Notificador y ProveedorRepuestos, realizadas por NotificadorCorreo y ProveedorRepuestosHttp. GestorMantenimiento depende solo de Notificador. Una forma de pago futura sería otra interfaz que extiende ServicioExterno, sin tocar el dominio.
(c) Estados y prioridades como enumeraciones	EstadoEquipo, EstadoOrden y Prioridad («enumeration»), con dependencias desde las clases que las usan.
Atributos con tipo, métodos y visibilidad	+ público, - privado, /equipo atributo derivado, cursiva para lo abstracto. Los tipos usan la notación de Python 3.10+ (str, date, Decimal).

Multiplicidades

Regla	Cómo se refleja en el diagrama	Multiplicidad
RN1	Un técnico responsable por orden y cero o más de apoyo. Solo el responsable inicia, suspende y finaliza: parámetro solicitante en esos métodos.	OrdenMantenimiento 0..* — 0..1 Tecnico (responsable, ver A1); 0..* — 0..* (apoyo)
RN2	No se puede expresar con una multiplicidad; se declara como restricción y se implementa en Tecnico.orden_programada_en().	Nota {RN2} en el diagrama
RN3	Una orden nace de un único reporte; un reporte origina como máximo una orden.	ReporteFalla 1 — 0..1 OrdenMantenimiento

Figura 1. Diagrama de clases UML de Maintenance Pro



5. Relaciones UML

Relación	*Tipo*	*Justificación*
OrdenMantenimiento — Actividad (1 a 0..*) (obligatoria)	Composición	La actividad es parte exclusiva de una orden: no existe sin ella, no se comparte con otra y se crea mediante registrar_actividad(). Si la orden desaparece, sus actividades también. El rombo relleno está en el lado de la orden (el "todo").
OrdenMantenimiento — RepuestoUtilizado	Composición	Mismo criterio: el uso de un repuesto solo tiene sentido dentro de la orden que lo consumió.
RepuestoUtilizado → Repuesto	Asociación dirigida	El repuesto del catálogo existe por sí mismo y es compartido por muchas órdenes. No es composición ni agregación porque el uso no "es parte" del repuesto. Solo se navega del uso hacia el catálogo.
Usuario ◁— Operador, Supervisor, Tecnico	Generalización	Relación "es un": los tres tipos comparten identificación, nombre y correo, y se diferencian por su rol. Con obtener_rol() abstracto cada subclase puede sustituir a Usuario (principio de sustitución de Liskov).
Notificador ◁⋯ NotificadorCorreo (y ProveedorRepuestos ◁- ProveedorRepuestosHttp) (servicios externos)	Realización	La clase concreta cumple el contrato de la interfaz. Agregar un proveedor SMS o una forma de pago significa crear una nueva clase que realiza la interfaz, sin modificar las clases del dominio (principio abierto/cerrado).
GestorMantenimiento → Notificador (servicios externos)	Asociación dirigida (uso)	El gestor mantiene una referencia al contrato Notificador y no a una clase concreta (inversión de dependencias); la implementación se inyecta al crearlo. No es composición porque el gestor no es dueño del servicio ni controla su ciclo de vida.
OrdenMantenimiento — Tecnico (responsable 0..1, apoyo 0..*)	Asociación	Los técnicos existen independientemente de las órdenes (atienden muchas), por lo que no hay relación todo-parte. Las dos asociaciones con rol distinguen al responsable (RN1) de los técnicos de apoyo.
ReporteFalla 1 — 0..1 OrdenMantenimiento	Asociación	Ambos son objetos con identidad propia. La multiplicidad codifica RN3.
Empresa — Equipo y Empresa — Usuario	Composición y agregación	Un equipo se registra siempre dentro de una empresa y pierde sentido sin ella (composición). Los usuarios se agrupan en la empresa pero son personas con existencia propia (agregación).
Equipo ⇢ EstadoEquipo, OrdenMantenimiento ⇢ EstadoOrden, ⇢ Prioridad	Dependencia	Las clases solo usan las enumeraciones como tipo de un atributo; no hay relación estructural.

PARTE C – Diagrama de secuencia

6. Escenario principal

Escenario: "Un supervisor selecciona un reporte de falla, genera una orden de mantenimiento, asigna un técnico y programa la fecha de atención". Participantes: Supervisor (actor), Sistema (GestorMantenimiento), ReporteFalla, OrdenMantenimiento, Tecnico y, para la notificación opcional, Notificador (interfaz).

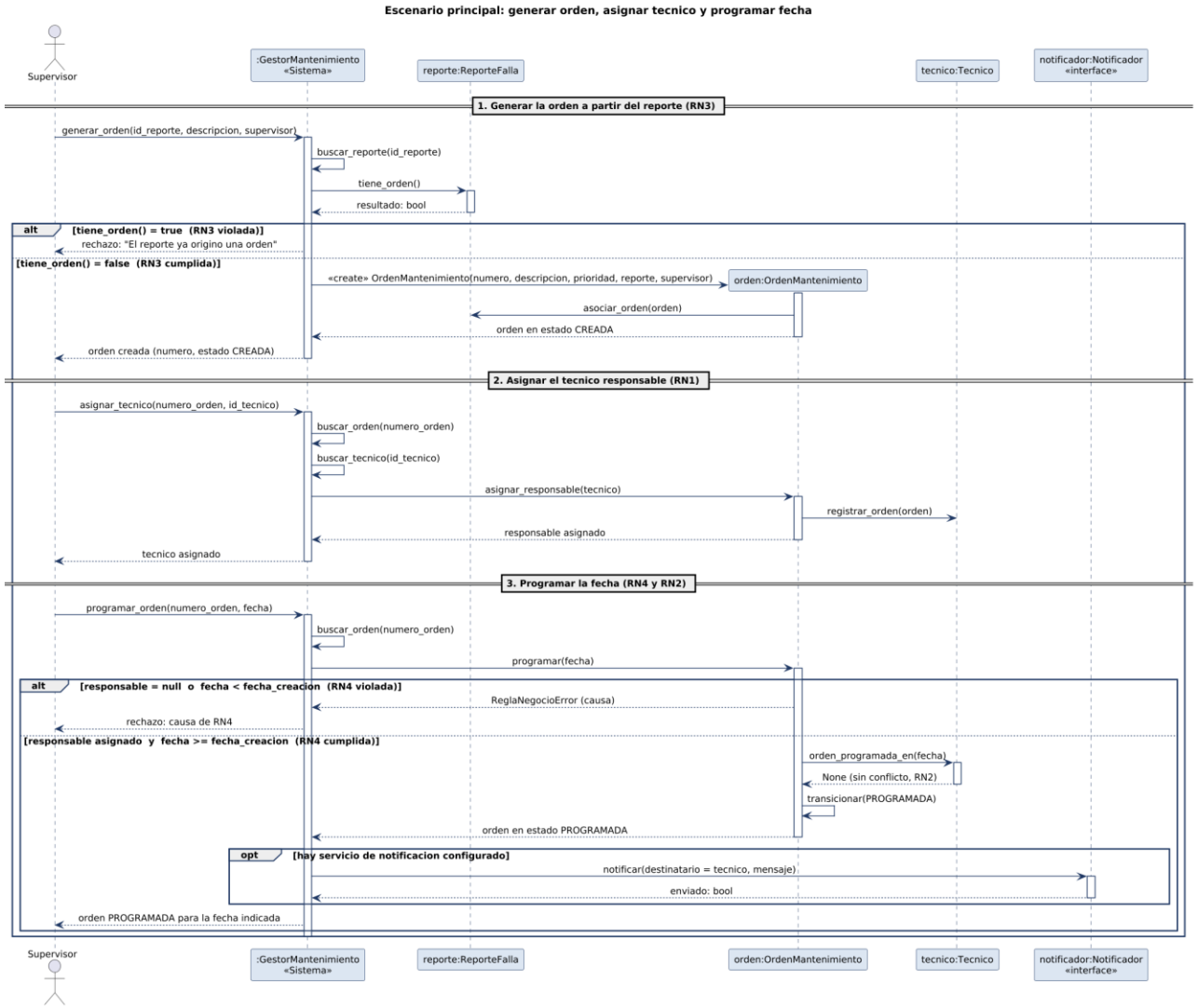


Figura 2.

Fragmentos combinados

Fragmento	Condición	Qué representa
alt (RN3)	tiene_orden() = true / false	Si el reporte ya originó una orden, el sistema rechaza la solicitud. Si no, crea la orden y esta se asocia al reporte (asociar_orden).
alt (RN4)	responsable = null o fecha < fecha_creacion / lo contrario	Si no hay responsable o la fecha es anterior a la creación, la orden rechaza la programación. Si se cumple, consulta la agenda del técnico y pasa a PROGRAMADA.
opt	hay servicio de notificación configurado	Solo si existe un Notificador, el sistema avisa al técnico. Es la abstracción de servicios externos en acción.

Mensajes y métodos del diagrama de clases

Mensaje	Clase receptora	Método en la Figura 1
generar_orden(id_reporte, descripcion, supervisor)	GestorMantenimiento	generar_orden(...)
buscar_reporte, buscar_orden, buscar_tecnico	GestorMantenimiento (auto-mensajes)	buscar_reporte(), buscar_orden(), buscar_tecnico()
tiene_orden()	ReporteFalla	tiene_orden()
«create» OrdenMantenimiento(...)	OrdenMantenimiento	constructor (__init__ en Python)
asociar_orden(orden)	ReporteFalla	asociar_orden()
asignar_tecnico(numero_orden, id_tecnico)	GestorMantenimiento	asignar_tecnico()
asignar_responsable(tecnico)	OrdenMantenimiento	asignar_responsable()
registrar_orden(orden)	Tecnico	registrar_orden()
programar_orden(numero_orden, fecha)	GestorMantenimiento	programar_orden()
programar(fecha)	OrdenMantenimiento	programar()
orden_programada_en(fecha)	Tecnico	orden_programada_en()
transicionar(PROGRAMADA)	OrdenMantenimiento (auto-mensaje)	-transicionar()
notificar(destinatario, mensaje)	Notificador	notificar()

7. Escenario alternativo (RN2)

El técnico elegido ya es responsable de otra orden programada para la misma fecha. El sistema rechaza la asignación y muestra la causa; la orden permanece en CREADA (A8).

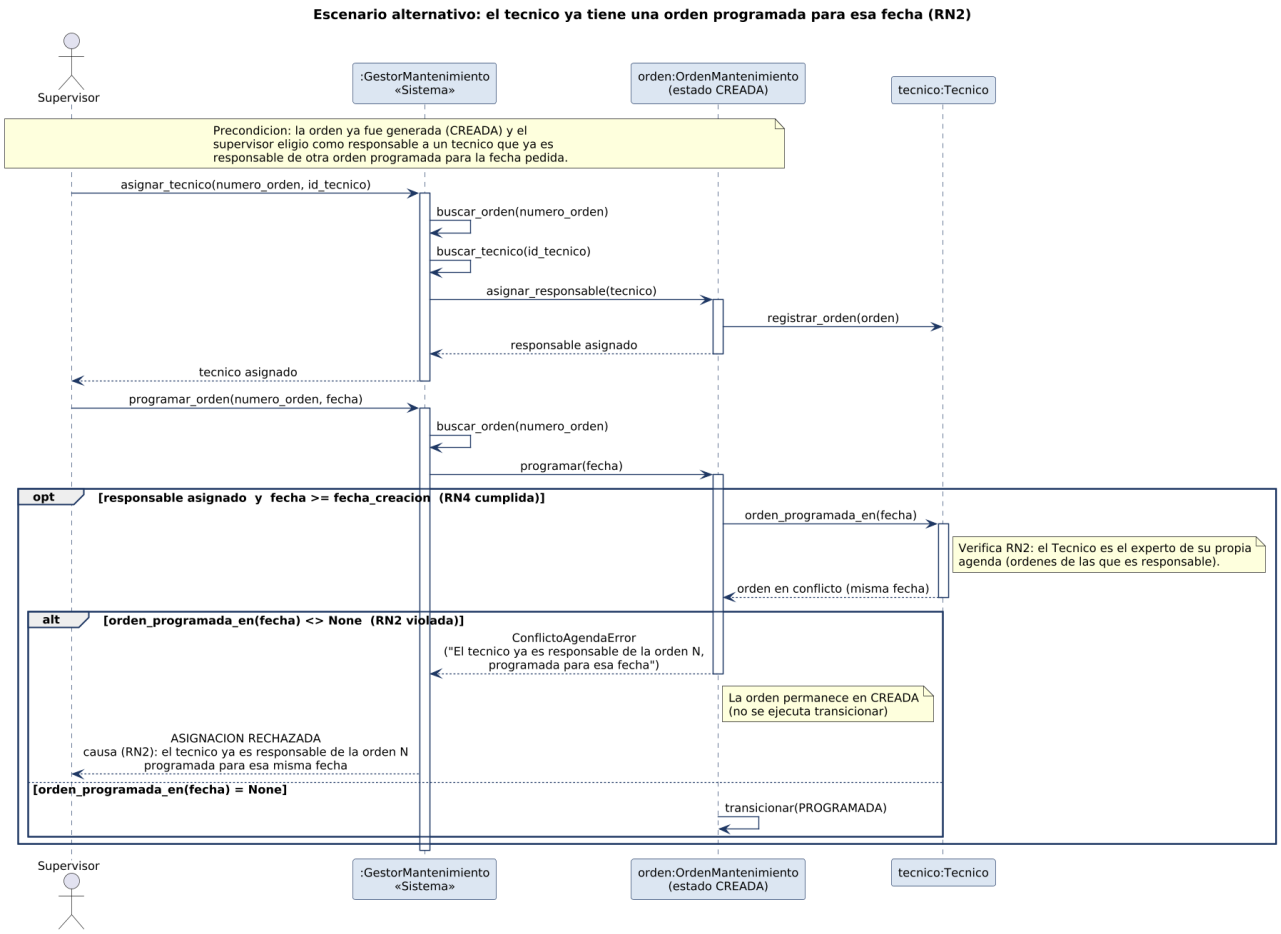


Figura 3.

¿Qué participante realiza la verificación y por qué?

La verificación la realiza el **`Tecnico`**, mediante orden_programada_en(fecha), invocado por OrdenMantenimiento.programar(). Las razones son:

- **Es el experto en la información** (patrón *Information Expert*): el técnico conoce las órdenes de las que es responsable, es decir, su propia agenda. Quien tiene los datos debe decidir sobre ellos.
- **La orden coordina, pero no conoce agendas ajenas.** OrdenMantenimiento conoce la fecha que se pide y es quien ejecuta la transición, por eso formula la pregunta al técnico; pero no inspecciona las demás órdenes.
- **El Sistema solo orquesta.** Si GestorMantenimiento revisara las agendas, acumularía lógica de negocio (un "controlador gordo") y duplicaría reglas que ya pertenecen al dominio. Su función aquí es traducir ConflictoAgendaError en un mensaje claro para el supervisor.
- **El Supervisor es un actor humano**; no ejecuta lógica del sistema.

La causa que se muestra al supervisor incluye el número de la orden en conflicto y la fecha, porque orden_programada_en() devuelve la orden que choca y no solo un booleano.

PARTE D – Máquina de estados

8. Estados de la orden

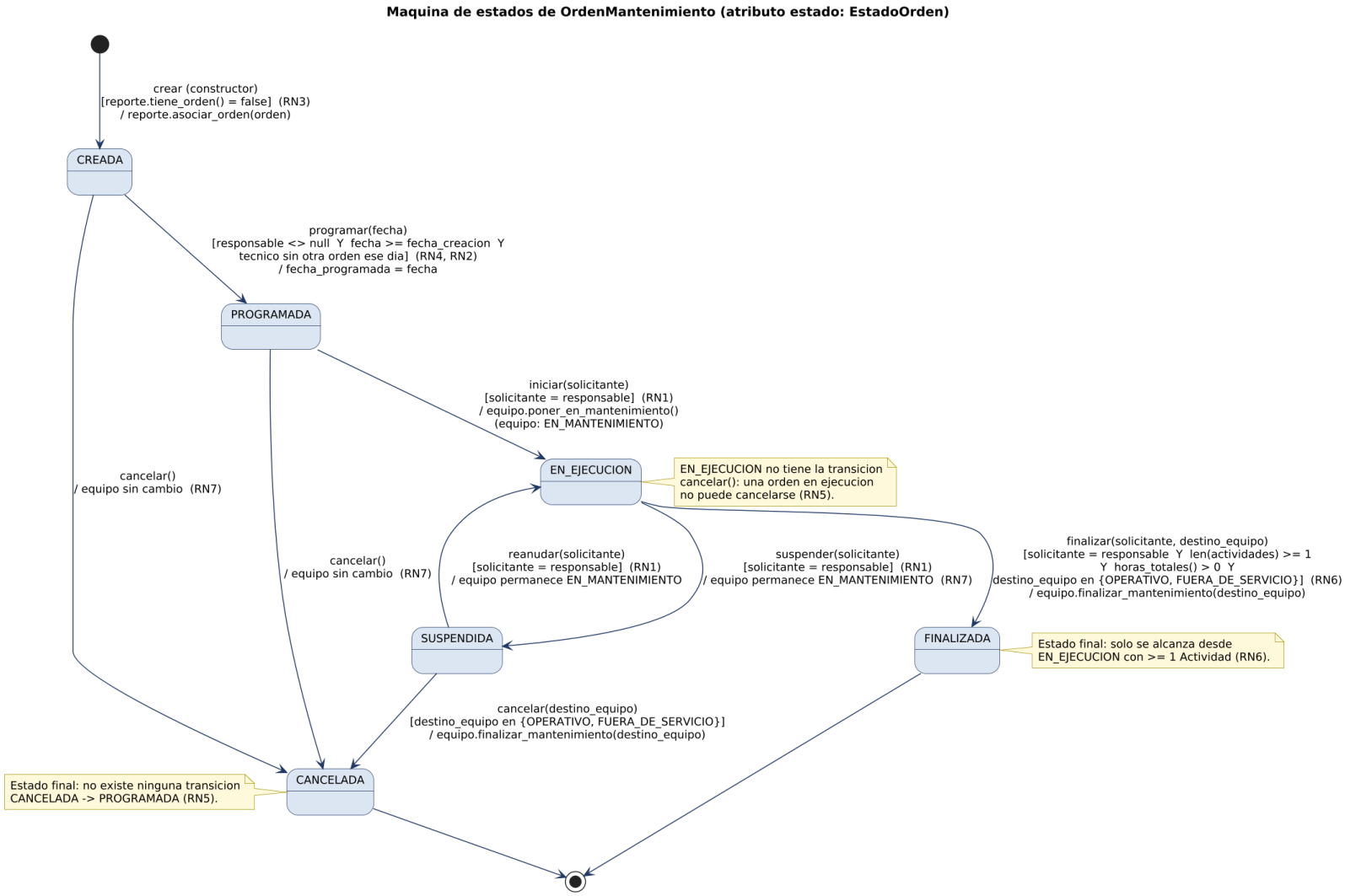


Figura 4.

Tabla de transiciones

Estado origen	Evento	Guarda	Acción (efecto sobre el equipo)	Estado destino
[inicial]	crear (constructor)	reporte.tiene_orden() = false (RN3)	reporte.asociar_orden(orden); equipo sin cambio	CREADA
CREADA	programar(fecha)	responsable ≠ null y fecha ≥ fecha_creacion y el técnico no tiene otra orden vigente ese día (RN4, RN2)	fecha_programada = fecha; equipo sin cambio	PROGRAMADA
CREADA	cancelar()	—	Equipo sin cambio (RN7)	CANCELADA
PROGRAMADA	cancelar()	—	Equipo sin cambio (RN7)	CANCELADA
PROGRAMADA	iniciar(solicitante)	solicitante = responsable (RN1)	equipo.poner_en_mantenimiento(): equipo → EN_MANTENIMIENTO	EN_EJECUCION
EN_EJECUCION	suspender(solicitante)	solicitante = responsable (RN1)	Equipo permanece EN_MANTENIMIENTO (RN7)	SUSPENDIDA
SUSPENDIDA	reanudar(solicitante)	solicitante = responsable (RN1, A5)	Equipo permanece EN_MANTENIMIENTO	EN_EJECUCION
SUSPENDIDA	cancelar(destino_equipo)	destino_equipo ∈ {OPERATIVO, FUERA_DE_SERVICIO} (A3)	equipo.finalizar_mantenimiento(destino_equipo): equipo → destino	CANCELADA
EN_EJECUCION	finalizar(solicitante, destino_equipo)	solicitante = responsable y len(actividades) ≥ 1 y horas_totales() > 0 (RN6) y destino_equipo ∈ {OPERATIVO, FUERA_DE_SERVICIO}	equipo.finalizar_mantenimiento(destino_equipo): equipo → OPERATIVO o FUERA_DE_SERVICIO (decide el supervisor)	FINALIZADA
FINALIZADA, CANCELADA	cualquier evento	—	Sin transición: se lanza TransicionInvalidaError (RN5)	(estado final)

Restricciones obligatorias

- **Una orden finalizada tiene al menos una actividad:** la única transición hacia FINALIZADA es finalizar() desde EN_EJECUCION y su guarda exige len(actividades) ≥ 1 y horas_totales() > 0 (RN6).
- **Una orden cancelada no puede volver a programarse:** CANCELADA es un estado final sin transiciones de salida, y programar() solo se admite desde CREADA (RN5). El código lanza TransicionInvalidaError.
- **Una orden suspendida puede volver a ejecución:** transición SUSPENDIDA → EN_EJECUCION con el evento reanudar(); el equipo permanece EN_MANTENIMIENTO.
- **Una orden EN_EJECUCION no puede cancelarse:** no existe la transición EN_EJECUCION → CANCELADA (RN5).

Coherencia con la clase. Los seis estados son los valores de EstadoOrden, el atributo estado guarda el estado actual y cada evento es un método público de OrdenMantenimiento (programar, iniciar, suspender, reanudar, finalizar, cancelar), más el constructor para la transición inicial.

PARTE E – Python y trazabilidad

9. Diseño orientado a objetos

Clase	Constructor y enumeraciones	Métodos relevantes
Equipo	__init__(codigo_interno, tipo, marca, modelo, fecha_adquisicion, estado=OPERATIVO); enumeración EstadoEquipo	poner_en_mantenimiento(), finalizar_mantenimiento(destino), esta_operativo()
OrdenMantenimiento	__init__(numero, descripcion, prioridad, reporte, supervisor, fecha_creacion); enumeraciones EstadoOrden y Prioridad; tabla _TRANSICIONES (RN5)	asignar_responsable(), agregar_apoyo(), programar(), iniciar(), suspender(), reanudar(), finalizar(), cancelar(), registrar_actividad(), agregar_repuesto(), horas_totales(), costo_total()
Actividad	__init__(descripcion, horas_trabajadas, tecnico, fecha); valida horas > 0	es_valida(), resumen()

10. Validación de reglas

finalizar() valida en este orden: transición permitida (RN5), permiso del responsable (RN1), actividades registradas (RN6) y horas totales positivas (RN6). Solo si todo se cumple cambia el estado y el equipo. La excepción propia es OrdenSinActividadesError:

```
class OrdenSinActividadesError(ReglaNegocioError):
    """RN6: no se puede finalizar una orden sin actividades registradas."""
```

```
    def finalizar(self, solicitante: Tecnico, destino_equipo: EstadoEquipo) -> None:
        """EN_EJECUCION -> FINALIZADA. RN6: requiere >= 1 actividad y horas totales > 0.
        destino_equipo es la decisión del supervisor (OPERATIVO o FUERA_DE_SERVICIO)."""
        self._validar_transicion(EstadoOrden.FINALIZADA)
        self._exigir_responsable(solicitante)
        if not self._actividades:
            raise OrdenSinActividadesError(
                f"No se puede finalizar la orden N° {self._numero}: "
                "no tiene actividades registradas (RN6)."
            )
        if self.horas_totales() <= 0:
            raise HorasTrabajadasInvalidasError(
                f"No se puede finalizar la orden N° {self._numero}: "
                "las horas trabajadas totales deben ser mayores que cero (RN6)."
            )
        self.equipo.finalizar_mantenimiento(destino_equipo)  # puede lanzar; la orden aún no cambió
        self._transicionar(EstadoOrden.FINALIZADA)
```

Ejecución: caso correcto y caso de error

El caso correcto es la orden 1 (finaliza bien); el caso de error es la orden 2 (se intenta finalizar sin actividades y la excepción deja la orden en EN_EJECUCION y el equipo sin cambios). Las secciones EXTRA muestran RN2, RN5 y RN3.

```
======================================================================
CASO CORRECTO: ciclo completo de la orden 1
======================================================================
1. OrdenMantenimiento(N° 1, CREADA) creada. Equipo: OPERATIVO
   [correo vía smtp.maintenancepro.local] Para ana@empresa.co: Se le asignó la orden N° 1 para el 2026-10-12.
2. OrdenMantenimiento(N° 1, PROGRAMADA), fecha programada: 2026-10-12
3. OrdenMantenimiento(N° 1, EN_EJECUCION). Equipo: EN_MANTENIMIENTO
4. Horas totales: 3.5 h | Costo total: $90,000 (no cambia aunque el catálogo suba, RN8)
5. OrdenMantenimiento(N° 1, FINALIZADA). Equipo: OPERATIVO

======================================================================
CASO DE ERROR (RN6): finalizar sin actividades registradas
```

```
========================================================================
   [correo vía smtp.maintenancepro.local] Para ana@empresa.co: Se le asignó la orden N° 2 para el 2026-10-14.
OrdenMantenimiento(N° 2, EN_EJECUCION). Equipo: EN_MANTENIMIENTO. Actividades registradas: 0
ERROR CAPTURADO [OrdenSinActividadesError]: No se puede finalizar la orden N° 2: no tiene actividades registradas
(RN6).
Estado tras el error: OrdenMantenimiento(N° 2, EN_EJECUCION) | Equipo: EN_MANTENIMIENTO (sin cambios)

========================================================================
EXTRA RN2: el técnico ya tiene una orden programada ese día
========================================================================
ERROR CAPTURADO [ConflictoAgendaError]: Ana Pérez ya es responsable de la orden N° 2, programada para el 2026-10-14
(RN2).
OrdenMantenimiento(N° 3, CREADA) (permanece CREADA)

========================================================================
EXTRA RN5: una orden EN_EJECUCION no puede cancelarse
========================================================================
ERROR CAPTURADO [TransicionInvalidaError]: Orden 2: transición no permitida EN_EJECUCION -> CANCELADA (RN5). Una
orden EN_EJECUCION no puede cancelarse.

========================================================================
EXTRA RN3: un reporte origina como máximo una orden
========================================================================
ERROR CAPTURADO [ReporteYaTieneOrdenError]: El reporte 1 ya originó una orden (RN3).

========================================================================
RECUPERACIÓN: se registra la actividad y la orden 2 ya puede finalizarse
========================================================================
OrdenMantenimiento(N° 2, FINALIZADA). Equipo: FUERA_DE_SERVICIO (decisión del supervisor)

========================================================================
EXTRA RN5: una orden CANCELADA no puede volver a PROGRAMADA
========================================================================
ERROR CAPTURADO [TransicionInvalidaError]: Orden 3: transición no permitida CANCELADA -> PROGRAMADA (RN5). CANCELADA
es un estado final.
```

11. Trazabilidad entre vistas

Cada fila relaciona una regla o requisito con su representación en el diagrama de clases, el de secuencia, la máquina de estados y el código.

N.º	Regla / requisito	Diagrama de clases	Diagrama de secuencia	Máquina de estados	Python
1	RN1 Responsable único; solo él inicia, suspende y finaliza	OrdenMantenimiento 0..* — 0..1 Tecnico (responsable); apoyo 0..*; asignar_responsable(), agregar_apoyo(); parámetro solicitante	Paso 2: asignar_responsable(tecnico)	Guarda solicitante = responsable en iniciar, suspender, reanudar y finalizar	asignar_responsable(), _exigir_responsable(), PermisoError
2	RN2 Un técnico no es responsable de dos órdenes programadas el mismo día	Tecnico.orden_programada_en(fecha), esta_disponible(); nota {RN2}	Figura 3: orden_programada_en(fecha) + alt de conflicto + ConflictoAgendaError	Guarda de CREADA → PROGRAMADA: técnico sin otra orden ese día	Tecnico.orden_programada_en(); programar() lanza ConflictoAgendaError
3	RN3 Un reporte origina como máximo una orden	ReporteFalla 1 — 0..1 OrdenMantenimiento; tiene_orden(), asociar_orden()	Figura 2: alt tiene_orden() = true / false; asociar_orden(orden)	Transición inicial crear con guarda reporte.tiene_orden() = false	ReporteFalla.asociar_orden(), GestorMantenimiento.generar_orden(), ReporteYaTieneOrdenError
4	RN4 PROGRAMADA solo con responsable y fecha ≥ creación	OrdenMantenimiento.programar(fecha); atributo fecha_programada	Figura 2: alt responsable = null o fecha < fecha_creacion	CREADA → PROGRAMADA, guarda responsable ≠ null y fecha ≥ fecha_creacion	OrdenMantenimiento.programar()
5	RN5 Transiciones permitidas; estados finales	Enumeración EstadoOrden; atributo estado; -validar_transicion(), -transicionar()	transicionar(PROGRAMADA) (auto-mensaje en programar)	Todas las transiciones; EN_EJECUCION sin cancelar; FINALIZADA y CANCELADA sin salida	_TRANSICIONES, _validar_transicion(), TransicionInvalidaError
6	RN6 Finalizar exige ≥ 1 actividad y horas > 0	finalizar(), registrar_actividad(), horas_totales(); composición OrdenMantenimiento ◆— Actividad 0..*	Fuera del escenario; se verifica en la máquina de estados	EN_EJECUCION → FINALIZADA, guarda len(actividades) ≥ 1 y horas_totales() > 0	finalizar() → OrdenSinActividadesError, HorasTrabajadasInvalidasError
7	RN7 Efecto sobre el equipo	Equipo.poner_en_mantenimiento(), finalizar_mantenimiento(destino); enum EstadoEquipo	—	Acciones / equipo.… en iniciar, finalizar y cancelar; suspender y reanudar sin cambio	Equipo.poner_en_mantenimiento(), finalizar_mantenimiento(); iniciar(), finalizar(), cancelar()

N.º	Regla / requisito	Diagrama de clases	Diagrama de secuencia	Máquina de estados	Python
8	RN8 Costo histórico del repuesto	Repuesto 1 ← 0..* RepuestoUtilizado; atributo costo_unitario; nota {RN8}	—	—	agregar_repuesto() copia repuesto.costo_referencia; el caso correcto muestra $90.000 estable
9	RN9 Costo de la orden y prioridades válidas	costo_total(), RepuestoUtilizado.subtotal(); enum Prioridad	—	—	costo_total(), subtotal(), Prioridad
10	Extensibilidad: servicios externos y formas de pago	ServicioExterno ◁ Notificador ◁⋯ NotificadorCorreo; GestorMantenimiento → Notificador	Figura 2: opt con notificar(destinatario, mensaje)	—	Notificador (clase abstracta), NotificadorCorreo, GestorMantenimiento.programar_orden()
11	Escenario: generar orden, asignar técnico y programar	GestorMantenimiento.generar_orden(), asignar_tecnico(), programar_orden()	Figura 2 completa	Transiciones crear y programar	Sección “CASO CORRECTO” de la ejecución

PARTE F – Análisis y justificación

12. Extensibilidad: InspectorExterno

Se agregan tres elementos nuevos y no se modifica ninguna clase existente (Figura 5):

- InspectorExterno, subclase de Usuario, que **no** hereda de Supervisor ni de Tecnico.
- Inspector, una interfaz pequeña con revisar_equipo(equipo, descripcion, severidad), realizada por InspectorExterno.
- ObservacionInspeccion (fecha, descripción, severidad), asociada a Equipo y a quien la generó. La dirección de la asociación va de la observación hacia el equipo, por eso Equipo no cambia.

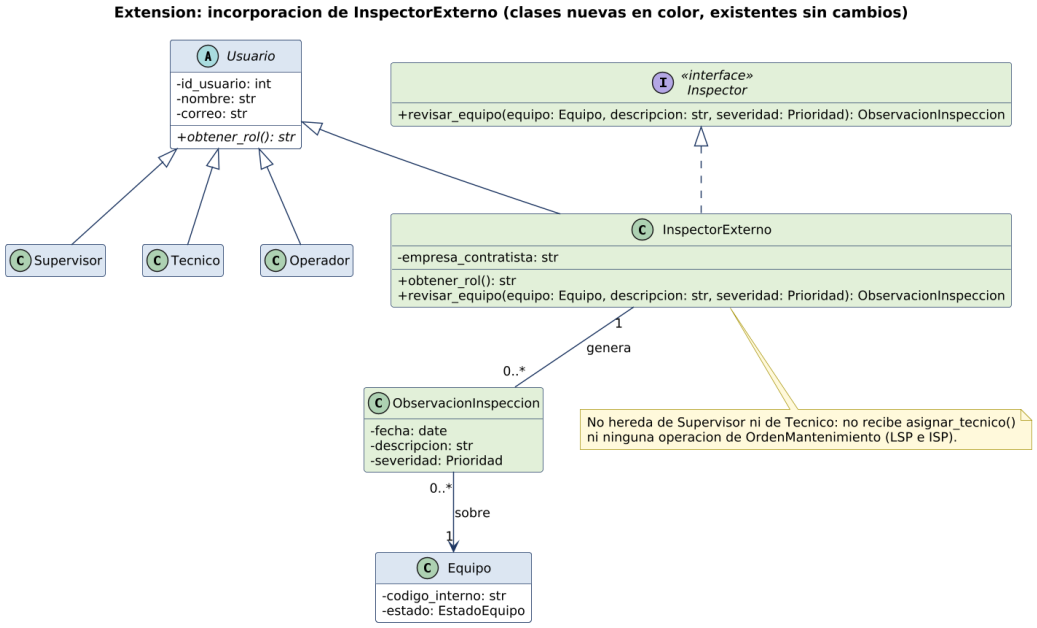


Figura 5.

Justificación

Alternativa	Evaluación
Heredar de Supervisor	Descartada. InspectorExterno heredaría revisar_reporte() y cancelar_orden() y quedaría un paso de poder asignar técnicos, que justamente no debe hacer. Violaría el principio de sustitución de Liskov: una subclase no puede quitar capacidades que la superclase promete.
Agregar un atributo rol a Usuario y verificar con if	Descartada. Obliga a modificar Usuario y cada método con permisos, y cada nuevo rol repite el cambio (viola abierto/cerrado).
Subclase de Usuario + interfaz Inspector + clase ObservacionInspeccion (elegida)	Herencia solo para lo que realmente es “un usuario”; interfaz para el comportamiento propio de inspeccionar; composición/asociación para los datos que genera. Como InspectorExterno no tiene métodos de asignación, no puede asignar técnicos por construcción.

Principios que respaldan la decisión: *abierto/cerrado* (se extiende agregando clases, no modificando Equipo, Supervisor ni OrdenMantenimiento); *segregación de interfaces* (Inspector expone solo lo que un inspector necesita,

no las operaciones de supervisión); *sustitución de Liskov* (un InspectorExterno puede usarse donde se espera un Usuario) y *responsabilidad única* (las observaciones tienen su propia clase en lugar de ensuciar Equipo).